OSLpay 商户入驻平台（Merchant Portal）TD

修订记录

版本	日期	修订内容
1.3	2024-03-04	1. 新增 MCP Server 设计（6 个工具，自动签名处理） 2. 新增结构化快速开始指南 API 设计 3. 新增 MCP 配置中心模块设计
1.2	2024-03-04	1. 新增快速开始页面设计（支持 WebSDK 和 OpenAPI 两种集成模式） 2. 新增菜单结构设计（支持二级菜单和权限控制） 3. 新增成员与权限管理模块设计 4. 新增域名白名单管理模块设计 5. 新增 API 请求日志模块设计 6. 完善 API 文档引擎（16 个接口，添加官方文档链接）
1.1	2024-03-03	1. 新增场景化 AI 提示词功能设计 2. 新增签名工具预填和自动传递功能 3. 新增全局环境切换设计 4. 新增草稿自动保存功能
1.0	2024-03-01	初始版本，定义系统架构和核心组件

概述

OSLpay 商户入驻平台采用前后端分离架构。前端使用 Next.js + React + TypeScript + Tailwind CSS 构建，后端使用 Java（JDK 25）+ Spring Boot 3.5.7 提供 RESTful API 服务。核心亮点是 API 文档引擎，它将 OpenAPI 3.0 规范文件解析为 AI 编程友好的交互式文档，支持多语言代码生成、在线签名验证和沙箱测试。

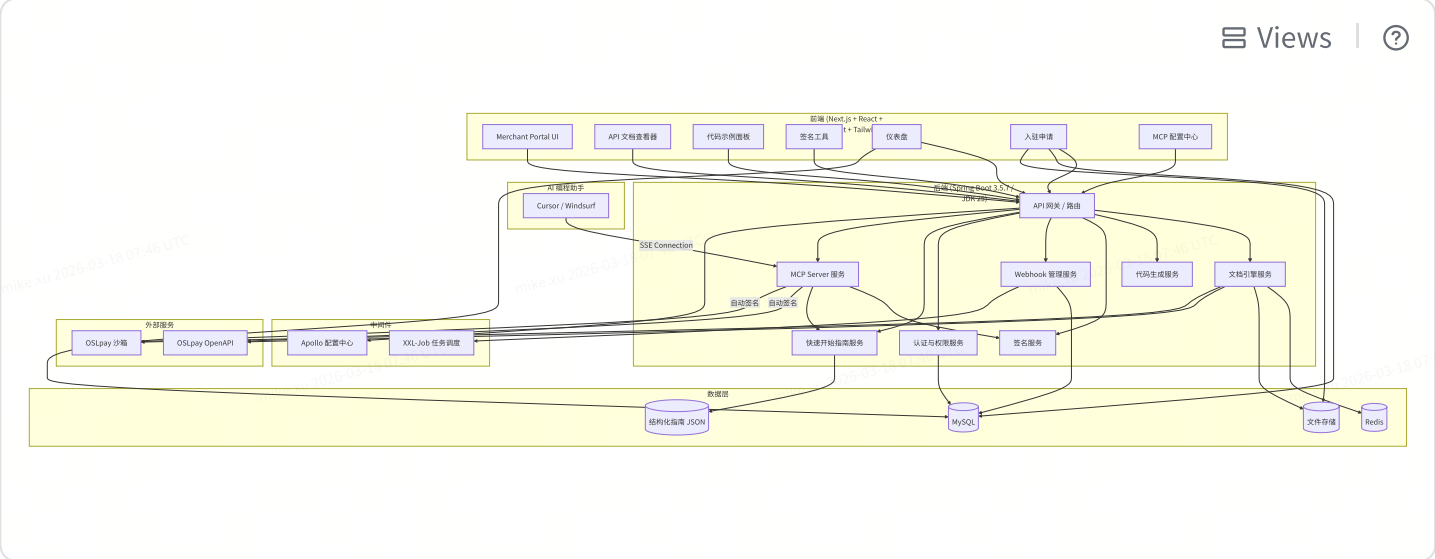
技术选型：

- Next.js + React + TypeScript + Tailwind CSS：SSR/SSG 支持、类型安全、原子化 CSS，适合构建复杂的交互式文档界面
- Java（JDK 25）+ Spring Boot 3.5.7：企业级后端框架，生态成熟，性能优异
- MySQL：存储商户数据、审计日志等结构化数据
- Redis：缓存 API 文档解析结果和会话信息
- Apollo：分布式配置中心，管理多环境配置（生产/沙箱端点地址、功能开关等）

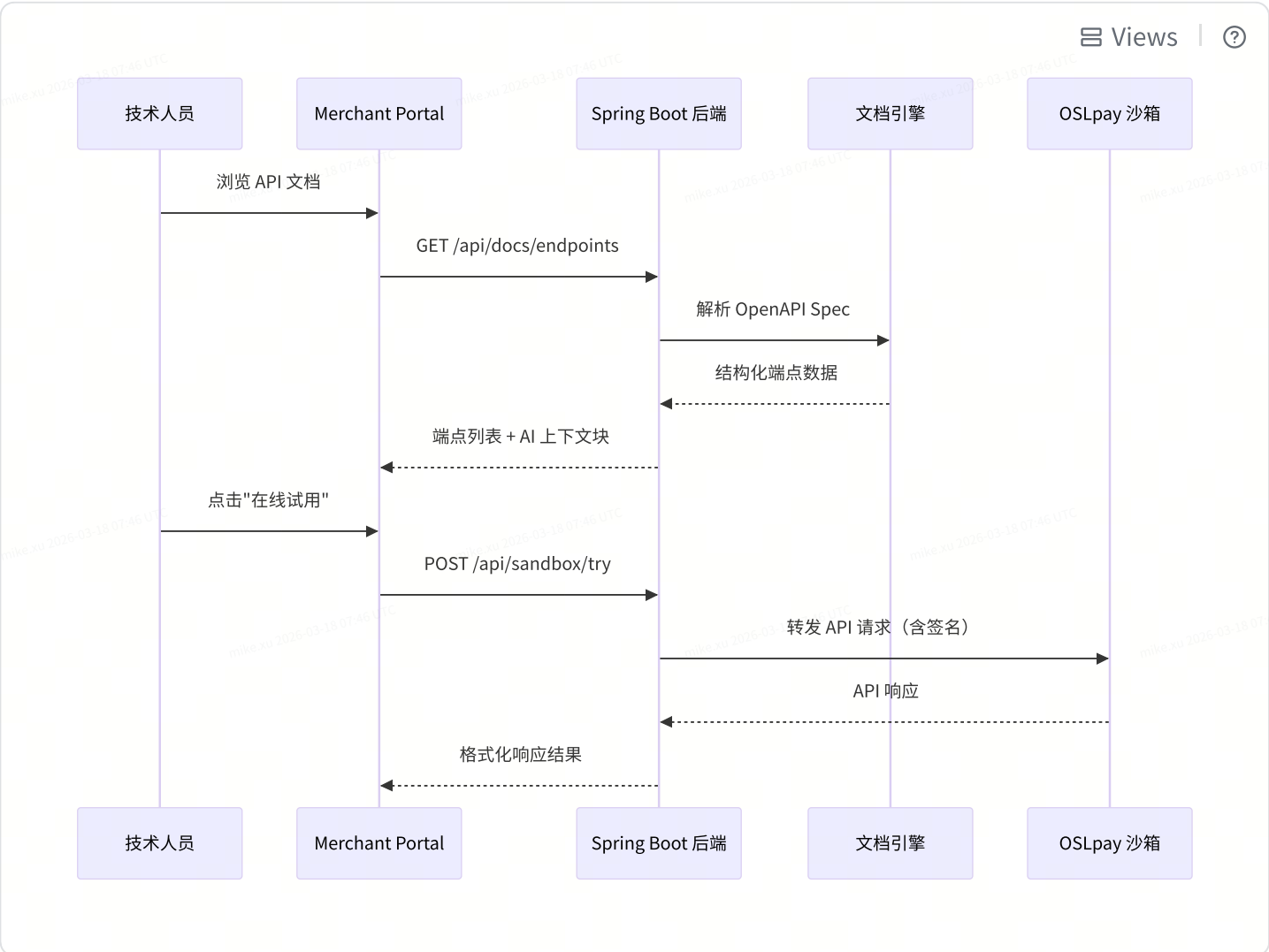
- XXL-Job：分布式任务调度，处理 Webhook 重试、数据报表生成等定时任务

架构

系统架构图



请求流程



组件与接口

1. API 文档引擎（DocEngine）

负责解析 OpenAPI 3.0 规范文件，生成结构化的端点数据和 AI 上下文块。后端使用 Spring Boot Controller + Service 实现。

Code block

```
// 以下接口定义使用 TypeScript 风格描述数据结构，
// 前端直接使用，后端用 Java Record/Class 对应实现

interface OpenAPISpec {
  raw: string;                    // 原始 YAML/JSON 内容
  parsed: ParsedSpec;             // 解析后的结构化数据
  version: string;                // 规范版本
  lastUpdated: Date;
}

interface ParsedSpec {
  info: SpecInfo;
  servers: ServerConfig[];
  endpoints: EndpointDefinition[];
  schemas: Record<string, SchemaDefinition>;
  errorCodes: ErrorCodeEntry[];
}

interface EndpointDefinition {
  path: string;                   // e.g. "/api/v1/quote"
  method: HttpMethod;
  summary: string;
  description: string;
  category: EndpointCategory;     // USER_MGMT | ORDER | QUOTE | CARD | CONFIG
  parameters: ParameterSchema[];
  requestBody?: RequestBodySchema;
  responses: Record<string, ResponseSchema>;
  dependencies: EndpointDependency[];  // 依赖的其他端点
  aiContextBlock: AIContextBlock;
}

interface AIContextBlock {
  endpoint: string;
  method: HttpMethod;
  purpose: string;
  inputSchema: JSONSchema;
  outputSchema: JSONSchema;
  dependencies: string[];         // 依赖端点路径列表
```

```
  errorCodes: ErrorCodeEntry[];
  exampleRequest: Record<string, unknown>;
  exampleResponse: Record<string, unknown>;
  notes: string[];                // AI 使用提示
}

interface ParameterSchema {
  name: string;
  in: 'query' | 'header' | 'path';
  required: boolean;
  type: string;
  enum?: string[];
  example?: unknown;
  description: string;
}

interface SchemaDefinition {
  name: string;
  type: string;
  properties: Record<string, PropertyDefinition>;
  required: string[];
  description: string;
}
```

REST API 接口：

方法	路径	描述
GET	`/api/docs/spec`	获取完整 OpenAPI 规范
GET	`/api/docs/endpoints`	获取端点列表（支持分类筛选）
GET	`/api/docs/endpoints/:path`	获取单个端点详情 + AI 上下文块
GET	`/api/docs/ai-context/:path`	获取端点的 AI 上下文块（纯文本）
GET	`/api/docs/schemas`	获取所有数据模型定义
GET	`/api/docs/error-codes`	获取完整错误码列表

2. 代码生成服务（CodeGen）

基于 OpenAPI 端点定义和模板引擎生成多语言代码示例。后端使用 Handlebars 模板引擎（通过 Java 的 handlebars.java 库）。

Code block

```
type SupportedLanguage = 'typescript' | 'python' | 'java' | 'javascript';

interface CodeGenerationRequest {
  endpointPath: string;
  language: SupportedLanguage;
  includeSignature: boolean;      // 是否包含签名计算代码
  environment: 'production' | 'sandbox';
}

interface CodeGenerationResult {
  language: SupportedLanguage;
  code: string;                   // 生成的代码
  dependencies: string[];         // 需要安装的依赖包
  notes: string[];                // 使用说明
}

interface CodeTemplate {
  language: SupportedLanguage;
  templateId: string;
  render(context: TemplateContext): string;
}

interface TemplateContext {
  endpoint: EndpointDefinition;
  baseUrl: string;
  signatureCode: string;          // 签名计算代码片段
  requestExample: Record<string, unknown>;
}
```

REST API 接口：

方法	路径	描述
POST	`/api/codegen/generate`	生成指定端点的代码示例
GET	`/api/codegen/languages`	获取支持的语言列表
POST	`/api/codegen/websdk`	生成 WebSDK 集成代码

3. 签名服务（SignService）

提供在线签名计算、验证和加密功能。

Code block

```
interface SignRequest {
```

```
  appId: string;
  timestamp: string;
  privateKey: string;              // PEM 或 Base64 格式
}

interface SignResult {
  signatureString: string;         // 签名原文: appId=[appId]&timestamp=[timestamp]
  signature: string;               // Base64 编码的签名值
  headers: {                       // 可直接使用的请求头
    appId: string;
    timestamp: string;
    signature: string;
  };
}

interface VerifyRequest {
  signatureString: string;
  signature: string;
  publicKey: string;
}

interface VerifyResult {
  valid: boolean;
  message: string;
}

interface EncryptRequest {
  plaintext: string;
  publicKey: string;               // Webhook 公钥
}

interface EncryptResult {
  ciphertext: string;              // Base64 编码的密文
}
```

REST API 接口：

方法	路径	描述
POST	`/api/sign/generate`	生成签名
POST	`/api/sign/verify`	验证签名
POST	`/api/sign/encrypt`	加密敏感字段

4. Webhook 管理服务（WebhookSvc）

管理 Webhook 端点配置、测试和日志。

Code block

```
interface WebhookConfig {
  id: string;
  merchantId: string;
  url: string;
  events: WebhookEventType[];
  secret: string;                    // 用于验签的密钥
  active: boolean;
  createdAt: Date;
  updatedAt: Date;
}

type WebhookEventType =
  | 'kyc_status_change'
  | 'defi_account_bind_status'
  | 'defi_account_auth_status'
  | 'order_status_change';

interface WebhookLog {
  id: string;
  configId: string;
  eventType: WebhookEventType;
  payload: Record<string, unknown>;
  responseStatus: number | null;
  responseBody: string | null;
  retryCount: number;
  status: 'success' | 'failed' | 'pending' | 'final_failed';
  createdAt: Date;
  lastRetryAt: Date | null;
}

interface WebhookTestRequest {
  configId: string;
  eventType: WebhookEventType;
}

interface WebhookTestResult {
  sent: boolean;
  responseStatus: number;
  responseBody: string;
  latencyMs: number;
}
```

REST API 接口：

方法	路径	描述
GET	`/api/webhooks`	获取 Webhook 配置列表
POST	`/api/webhooks`	创建 Webhook 配置
PUT	`/api/webhooks/:id`	更新 Webhook 配置
DELETE	`/api/webhooks/:id`	删除 Webhook 配置
POST	`/api/webhooks/:id/test`	测试 Webhook
GET	`/api/webhooks/:id/logs`	获取 Webhook 推送日志

5. 认证与权限服务（AuthService）

处理用户登录、角色管理和权限控制。

Code block

```
type UserRole = 'business' | 'technical' | 'admin';

interface User {
  id: string;
  merchantId: string;
  email: string;
  name: string;
  role: UserRole;
  createdAt: Date;
  lastLoginAt: Date;
}

interface AuthToken {
  accessToken: string;
  refreshToken: string;
  expiresIn: number;
}

interface AuditLog {
  id: string;
  userId: string;
  action: string;
  resource: string;
  result: 'success' | 'failure';
  details: Record<string, unknown>;
```

```
  ipAddress: string;
  createdAt: Date;
}

// 角色权限映射
const ROLE_PERMISSIONS: Record<UserRole, string[]> = {
  business: ['onboarding:read', 'onboarding:write', 'dashboard:read', 'dashboard:export'],
  technical: ['docs:read', 'codegen:use', 'sign:use', 'webhook:manage', 'sandbox:use', 'developer:read'],
  admin: ['*'],
};
```

REST API 接口：

方法	路径	描述
POST	`/api/auth/login`	用户登录
POST	`/api/auth/refresh`	刷新令牌
GET	`/api/auth/me`	获取当前用户信息
GET	`/api/audit-logs`	获取审计日志

6. 商户入驻模块（Onboarding）

处理商户入驻申请的提交和状态追踪。

Code block

```
type OnboardingStatus = 'draft' | 'submitted' | 'under_review' | 'info_requested' | 'approved' | 'rejected';

interface OnboardingApplication {
  id: string;
  merchantId: string;
  status: OnboardingStatus;
  companyInfo: CompanyInfo;
  businessInfo: BusinessInfo;
  documents: UploadedDocument[];
  submittedAt: Date | null;
  reviewNotes: string[];
  currentStep: number;
  totalSteps: number;
}
```

```typescript

interface CompanyInfo {
  legalName: string;
  registrationNumber: string;
  country: string;
  address: string;
  website: string;
}

interface BusinessInfo {
  businessType: string;
  expectedVolume: string;
  integrationMode: 'websdk' | 'openapi' | 'both';
  targetMarkets: string[];
}

interface UploadedDocument {
  id: string;
  type: DocumentType;
  fileName: string;
  fileSize: number;              // bytes
  mimeType: 'application/pdf' | 'image/jpeg' | 'image/png';
  uploadedAt: Date;
  status: 'pending' | 'approved' | 'rejected';
}

type DocumentType = 'business_license' | 'id_proof' | 'address_proof' | 'compliance_cert';
```

REST API 接口：

方法	路径	描述
GET	`/api/onboarding/application`	获取当前申请
PUT	`/api/onboarding/application`	更新申请信息
POST	`/api/onboarding/application/submit`	提交申请
POST	`/api/onboarding/documents`	上传文件
GET	`/api/onboarding/status`	获取申请状态

7. 仪表盘模块（Dashboard）

展示交易数据和订单管理。

Code block

```
interface DashboardMetrics {
  totalVolume: number;            // 交易总额
  successRate: number;            // 成功率 (0-1)
  orderCount: number;             // 订单数量
  activeUsers: number;            // 活跃用户数
  period: DateRange;
}

interface DateRange {
  start: Date;
  end: Date;
}

type OrderStatus = 'CREATEED' | 'PROCESSING' | 'CONVERSION_FAILED' | 'CHAIN_WITHDRAW_WAIT' | 'SUCCESSED' | 'COMPLETED' | 'FAILED';
type PaymentMethod = 'CARD' | 'GOOGLEPAY' | 'APPLE_PAY';

interface OrderListQuery {
  status?: OrderStatus;
  paymentMethod?: PaymentMethod;
  dateRange?: DateRange;
  page: number;
  pageSize: number;
}

interface OrderSummary {
  orderId: string;
  merchantOrder: string;
  status: OrderStatus;
  subStatus?: 'WITHDRAWAL_SUCCESS' | 'WITHDRAWAL_FAILED';
  amount: number;
  fiatCurrency: string;
  crypto: string;
  network: string;
  paymentMethod: PaymentMethod;
  createdAt: Date;
  updatedAt: Date;
}

interface OrderDetail extends OrderSummary {
  statusHistory: StatusTransition[];
  paymentInfo: PaymentInfo;
```

```
  chainInfo?: ChainInfo;
}

interface StatusTransition {
  from: OrderStatus;
  to: OrderStatus;
  timestamp: Date;
  reason?: string;
}
```

REST API 接口：

方法	路径	描述
GET	`/api/dashboard/metrics`	获取关键指标
GET	`/api/dashboard/orders`	获取订单列表
GET	`/api/dashboard/orders/:id`	获取订单详情
GET	`/api/dashboard/export`	导出 CSV

8. 快速开始模块（GettingStarted）

提供集成指南和快速开始流程。

Code block

```
type IntegrationMode = 'websdk' | 'api';

interface GettingStartedContent {
  mode: IntegrationMode;
  steps: IntegrationStep[];
  quickLinks: QuickLink[];
}

interface IntegrationStep {
  number: number;
  title: string;
  description: string;
  action?: React.ReactNode;
}

interface QuickLink {
  icon: React.ReactNode;
  title: string;
```

```
    description: string;
    href: string;
  }
```

功能特性：

- 支持 WebSDK 和 OpenAPI 两种集成模式切换
- WebSDK 模式：4 步流程（访问测试页面、填写 KYC、使用测试卡号、查看货币网络）
- OpenAPI 模式：4 步流程（获取凭证、实现签名、调用 API、配置 Webhook）+ 签名加密指南
- 提供快速链接导航到 API 文档、签名工具、Webhook 管理
- 显示支持联系方式

9. 菜单导航模块（Navigation）

提供层级化的菜单结构和权限控制。

Code block

```
interface NavItem {
  label: string;
  href?: string;
  icon: string;
  roles: UserRole[];
  subItems?: SubMenuItem[];
}

interface SubMenuItem {
  label: string;
  href: string;
  roles: UserRole[];
}

const NAV_ITEMS: NavItem[] = [
  {
    label: "快速开始",
    href: "/getting-started",
    icon: "rocket",
    roles: ["business", "technical", "admin"],
  },
  {
    label: "仪表盘",
    href: "/dashboard",
    icon: "chart",
    roles: ["business", "admin"],
  },
```

```
  {
    label: "业务管理",
    icon: "briefcase",
    roles: ["business", "admin"],
    subItems: [
      { label: "入驻申请", href: "/onboarding", roles: ["business", "admin"] },
      { label: "成员与权限", href: "/team", roles: ["admin"] },
    ],
  },
  {
    label: "开发者套件",
    icon: "code",
    roles: ["technical", "admin"],
    subItems: [
      { label: "API 文档", href: "/docs", roles: ["technical", "admin"] },
      { label: "API 凭证", href: "/developer", roles: ["technical", "admin"] },
      { label: "域名白名单", href: "/whitelist", roles: ["technical", "admin"] },
      { label: "Webhook 管理", href: "/webhooks", roles: ["technical", "admin"] },
      { label: "签名工具", href: "/sign-tools", roles: ["technical", "admin"] },
      { label: "API 请求日志", href: "/logs", roles: ["technical", "admin"] },
    ],
  },
];

function getNavItemsForRole(role: UserRole): NavItem[] {
  return NAV_ITEMS.filter(item => {
    if (!item.roles.includes(role)) return false;
    if (item.subItems) {
      const visibleSubItems = item.subItems.filter(sub => sub.roles.includes(role));
      return visibleSubItems.length > 0;
    }
    return true;
  }).map(item => {
    if (item.subItems) {
      return {
        ...item,
        subItems: item.subItems.filter(sub => sub.roles.includes(role))
      };
    }
    return item;
  });
}
```

功能特性：

- 支持一级菜单和二级菜单
- 基于角色的菜单过滤
- 自动展开包含当前路径的菜单
- 激活状态高亮（黑色背景 + 荧光绿图标）

10. 成员与权限管理模块（TeamManagement）

管理团队成员和角色权限。

Code block

```
interface TeamMember {
  id: string;
  name: string;
  email: string;
  role: "admin" | "business" | "technical";
  status: "active" | "pending";
  createdAt: string;
}

const ROLE_LABELS = {
  admin: "管理员",
  business: "业务人员",
  technical: "技术开发",
};

const ROLE_DESCRIPTIONS = {
  admin: "全部权限",
  business: "仅看申请/账单",
  technical: "仅看开发者套件",
};
```

REST API 接口：

方法	路径	描述
GET	`/api/team/members`	获取成员列表
POST	`/api/team/members`	邀请新成员
DELETE	`/api/team/members/:id`	移除成员
PUT		更新成员角色

	`/api/team/members/:id/role`	

11. 域名白名单管理模块（WhitelistManagement）

管理 Web SDK 来源校验的域名白名单。

Code block

```
interface WhitelistDomain {
  id: string;
  domain: string;
  status: "active" | "pending";
  createdAt: string;
}

function validateDomain(domain: string): boolean {
  try {
    const url = new URL(domain);
    return url.protocol === "http:" || url.protocol === "https:";
  } catch {
    return false;
  }
}
```

REST API 接口：

方法	路径	描述
GET	`/api/whitelist/domains`	获取域名列表
POST	`/api/whitelist/domains`	添加域名
DELETE	`/api/whitelist/domains/:id`	移除域名

12. API 请求日志模块（ApiLogs）

展示最近的 API 请求日志，用于排查集成错误。

Code block

```
interface ApiLog {
  id: string;
  timestamp: string;
  method: string;
```

```
  path: string;
  statusCode: number;
  duration: number;
  requestBody?: string;
  responseBody?: string;
}

const METHOD_COLORS: Record<string, string> = {
  GET: "bg-green-100 text-green-800",
  POST: "bg-blue-100 text-blue-800",
  PUT: "bg-yellow-100 text-yellow-800",
  DELETE: "bg-red-100 text-red-800",
};

function getStatusColor(statusCode: number): string {
  if (statusCode >= 200 && statusCode < 300) return "text-green-600";
  if (statusCode >= 400 && statusCode < 500) return "text-yellow-600";
  if (statusCode >= 500) return "text-red-600";
  return "text-gray-600";
}
```

REST API 接口：

方法	路径	描述
GET	`/api/logs/requests`	获取最新 10 条请求日志
GET	`/api/logs/requests/:id`	获取日志详情

13. MCP Server（远程服务）

提供 Model Context Protocol 服务，让 AI 编程助手能够通过 SSE (Server-Sent Events) 远程调用 OSL Pay API。

Code block

```
// MCP Server 工具定义
interface McpTool {
  name: string;
  description: string;
  inputSchema: JSONSchema;
}

const MCP_TOOLS: McpTool[] = [
  {
```

```
10        name: "oslpay_get_quote",
11        description: "获取法币到加密货币的报价",
12        inputSchema: {
13          type: "object",
14          properties: {
15            fiatAmount: { type: "number", description: "法币金额" },
16            fiatCurrency: { type: "string", description: "法币币种
    (USD/EUR/GBP/HKD) " },
17            cryptoCurrency: { type: "string", description: "加密货币
    (BTC/ETH/USDT/USDC) " },
18            network: { type: "string", description: "网络
    (ERC20/TRC20/BTC/Polygon) " },
19          },
20          required: ["fiatAmount", "fiatCurrency", "cryptoCurrency", "network"],
21        },
22      },
23      {
24        name: "oslpay_create_order",
25        description: "创建支付订单",
26        inputSchema: {
27          type: "object",
28          properties: {
29            quoteId: { type: "string", description: "报价 ID" },
30            merchantOrderNo: { type: "string", description: "商户订单号" },
31            redirectUrl: { type: "string", description: "支付完成后的重定向 URL" },
32          },
33          required: ["quoteId", "merchantOrderNo"],
34        },
35      },
36      {
37        name: "oslpay_query_order",
38        description: "查询订单状态",
39        inputSchema: {
40          type: "object",
41          properties: {
42            orderId: { type: "string", description: "订单 ID" },
43          },
44          required: ["orderId"],
45        },
46      },
47      {
48        name: "oslpay_generate_signature",
49        description: "生成 API 请求签名（RSA SHA256withRSA）",
50        inputSchema: {
51          type: "object",
52          properties: {
53            appId: { type: "string", description: "应用 ID" },
```

```
54          timestamp: { type: "string", description: "Unix 时间戳（秒）" },
55          privateKey: { type: "string", description: "RSA 私钥（PEM 格式）" },
56        },
57        required: ["appId", "timestamp", "privateKey"],
58      },
59    },
60    {
61      name: "oslpay_get_currency_list",
62      description: "获取支持的加密货币列表",
63      inputSchema: {
64        type: "object",
65        properties: {},
66      },
67    },
68    {
69      name: "oslpay_get_guide",
70      description: "获取快速开始指南（AI 可读的结构化数据）",
71      inputSchema: {
72        type: "object",
73        properties: {
74          mode: {
75            type: "string",
76            enum: ["websdk", "openapi"],
77            description: "集成模式"
78          },
79        },
80      },
81    },
82  ];
83
84  // MCP Server 远程配置
85  interface McpServerConfig {
86    url: string;                    // MCP Server 远程地址
87    transport: {
88      type: "sse";                  // Server-Sent Events
89    };
90    env?: {
91      OSLPAY_ENV?: "production" | "sandbox";
92    };
93  }
94
95  // 签名处理逻辑
96  interface SignatureHandler {
97    generateSignature(appId: string, timestamp: string, privateKey: string):
    string;
98    addAuthHeaders(request: HttpRequest, appId: string, privateKey: string):
    HttpRequest;
```

```
}

// MCP Server 端点
interface McpEndpoints {
  production: string;          // "https://mcp.osl-pay.com/sse"
  sandbox: string;             // "https://mcptest.osl-pay.com/sse"
}
```

核心功能:

- 自动生成 RSA SHA256withRSA 签名
- 自动添加请求头（X-App-Id, X-Timestamp, X-Signature）
- 封装 6 个核心工具（报价、订单、查询、签名、货币列表、快速指南）
- 智能处理 API 调用顺序（先报价后订单）
- 支持生产/沙箱环境切换

部署方式:

- 部署为远程服务
- 生产环境：`https://mcp.osl-pay.com/sse`
- 沙箱环境：`https://mcptest.osl-pay.com/sse`
- 使用 SSE (Server-Sent Events) 协议通信

技术实现:

- Spring Boot 实现（与主后端集成）
- 使用 `@modelcontextprotocol/sdk` 协议规范
- SSE 端点处理 MCP 协议消息
- 复用现有的签名服务和 API 客户端

REST API 接口：

方法	路径	描述
GET	`/mcp/sse`	MCP Server SSE 端点（生产环境）
GET	`/mcp/sse?env=sandbox`	MCP Server SSE 端点（沙箱环境）
GET	`/mcp/tools`	获取可用工具列表
POST	`/mcp/tools/:toolName`	直接调用工具（用于测试）

14. 结构化快速开始指南 API

提供 JSON 格式的结构化指南数据，供 AI 和前端使用。

Code block

```
interface GettingStartedGuide {
  version: string;
  lastUpdated: string;
  modes: {
    websdk: IntegrationMode;
    openapi: IntegrationMode;
  };
  commonInfo: CommonInfo;
}

interface IntegrationMode {
  title: string;
  description: string;
  steps: IntegrationStep[];
  codeExamples?: CodeExample[];
  faqs?: FAQ[];
}

interface IntegrationStep {
  number: number;
  title: string;
  description: string;
  details?: string[];
  codeSnippet?: string;
  expectedResult?: string;
  troubleshooting?: string[];
}

interface CodeExample {
  language: "nodejs" | "python" | "java";
  title: string;
  code: string;
  dependencies: string[];
  notes: string[];
}

interface FAQ {
  question: string;
  answer: string;
  relatedSteps?: number[];
}

```

```typescript
interface CommonInfo {
  supportedCurrencies: {
    fiat: string[];
    crypto: string[];
  };
  supportedNetworks: string[];
  testCards: TestCard[];
  contactInfo: {
    email: string;
    website: string;
  };
}

interface TestCard {
  type: string;
  number: string;
  expiry: string;
  cvv: string;
  scenario: string;
}
```

REST API 接口：

方法	路径	描述
GET	`/api/guides/getting-started`	获取完整的快速开始指南
GET	`/api/guides/getting-started/:mode`	获取特定模式的指南（websdk/openapi）

数据存储:

- JSON 文件：`backend/src/main/resources/guides/getting-started.json`
- 包含两种集成模式的详细步骤
- 包含多语言代码示例（Node.js、Python、Java）
- 包含常见问题和解决方案

AI 使用场景:

- MCP Server 调用 `oslpay_get_guide` 工具获取指南
- AI 根据开发者技术栈推荐集成模式
- AI 生成完整的集成代码

- AI 提供测试步骤和测试数据

15. MCP 配置中心模块

提供 MCP 配置管理和展示功能。

Code block

```
interface McpConfigCenterProps {
  environment: "production" | "sandbox";
}

interface McpConfigDisplay {
  config: McpServerConfig;
  tools: McpToolInfo[];
  setupSteps: SetupStep[];
}

interface McpToolInfo {
  name: string;
  description: string;
}

interface SetupStep {
  number: number;
  title: string;
  description: React.ReactNode;
}

function generateMcpConfig(environment: "production" | "sandbox"): McpServerConfig {
  const mcpUrl = environment === "production"
    ? "https://mcp.osl-pay.com/sse"
    : "https://mcptest.osl-pay.com/sse";

  return {
    mcpServers: {
      oslpay: {
        url: mcpUrl,
        transport: {
          type: "sse",
        },
      },
    },
  };
}
```

功能特性:

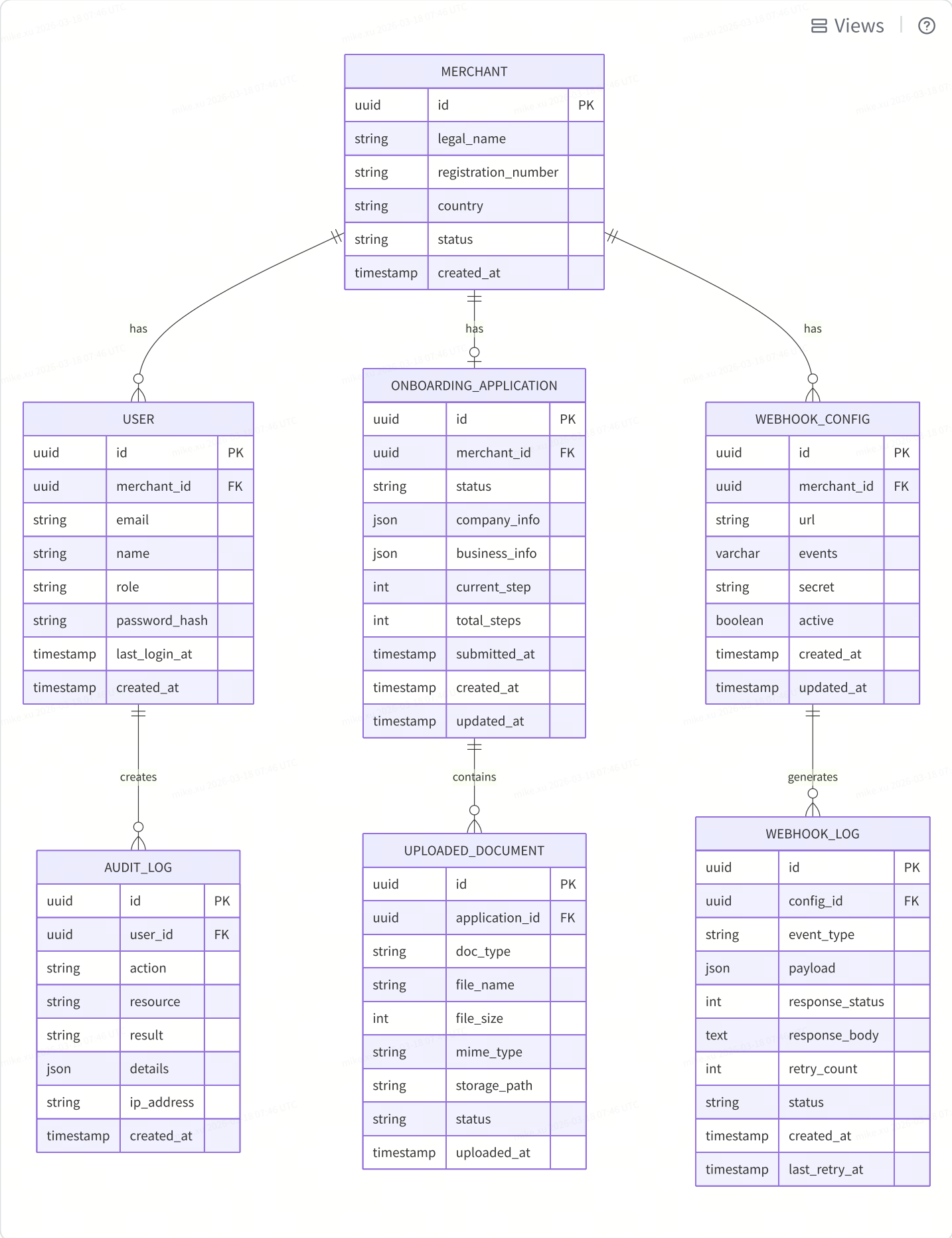
- 环境感知（自动识别生产/沙箱环境）
- 一键复制配置
- 展示可用工具列表（6 个工具）
- 提供配置步骤指南（3 步）
- 显示支持联系方式

配置步骤:

1. 复制 MCP 配置（包含远程 SSE 端点 URL）
2. 打开 AI 编程助手的配置文件并粘贴
3. 开始使用（AI 助手通过 SSE 连接到远程 MCP Server）

数据模型

数据库 ER 图



OpenAPI 规范存储

OpenAPI 规范文件以 YAML 格式存储在文件系统中，解析后的结构化数据缓存在 Redis 中。

Code block

```
/specs/
    ├── oslpay-openapi.yaml          # 主规范文件
    ├── schemas/                      # 可复用的 Schema 定义
    │   ├── order.yaml
    │   ├── user.yaml
    │   └── card.yaml
    └── examples/                     # 请求/响应示例
        ├── quote-request.json
        └── quote-response.json
```

代码模板存储

代码生成模板按语言分类存储：

Code block

```
/templates/
    ├── typescript/
    │   ├── api-call.hbs              # API 调用模板
    │   ├── signature.hbs             # 签名计算模板
    │   └── websdk.hbs                # WebSDK 集成模板
    ├── python/
    │   ├── api-call.hbs
    │   └── signature.hbs
    └── java/
        ├── api-call.hbs
        └── signature.hbs
```

正确性属性

属性（Property）是一种在系统所有有效执行中都应成立的特征或行为——本质上是关于系统应该做什么的形式化陈述。属性是人类可读规范与机器可验证正确性保证之间的桥梁。

以下属性基于需求文档中的验收标准推导而来，每个属性都包含明确的"对于任意"（for all）量化陈述，适合用属性基测试（Property-Based Testing）验证。

Property 1: OpenAPI 规范有效性

*对于任意*由 API_Doc_Engine 生成的规范文件，该文件应能通过 OpenAPI 3.0 标准验证器的校验，不产生任何错误。

验证: 需求 1.1

Property 2: 端点定义完整性

对于任意 OpenAPI 规范中的端点定义，解析后的结构应同时包含完整的请求参数 Schema（字段名、类型、是否必填、枚举值、示例值、描述）和完整的响应 Schema（成功响应和错误响应的字段名、类型、示例值、描述）。

验证: 需求 1.2, 1.3

Property 3: Schema 引用完整性

对于任意 OpenAPI 规范文件中的 `$ref` 引用，该引用指向的 Schema 应在 `components/schemas` 中存在且定义完整。

验证: 需求 1.4

Property 4: 错误码定义完整性

*对于任意*错误码条目，应包含错误码值、错误消息和处理建议三个必要字段，且所有字段非空。

验证: 需求 1.6

Property 5: 端点分类筛选正确性

*对于任意*分类筛选条件，筛选后返回的端点列表中每个端点的分类属性应与筛选条件一致。

验证: 需求 2.1

Property 6: 端点详情与 AI 上下文块完整性

*对于任意*端点定义，生成的详情视图和 AI_Context_Block 应包含所有必要字段：路径、HTTP 方法、参数 Schema、响应 Schema、依赖关系列表和错误码列表。

验证: 需求 2.2, 2.3

Property 7: AI 上下文块格式化 Round-Trip

对于任意 AI_Context_Block 对象，将其格式化为 AI 提示词文本后，该文本应包含原始对象中的所有关键信息（端点路径、方法、输入 Schema、输出 Schema）。

验证: 需求 2.4

Property 8: 多语言代码生成完整性

*对于任意*端点定义和任意支持的编程语言（TypeScript、Python、Java），生成的代码示例应包含请求构造逻辑、HTTP 调用逻辑、响应解析逻辑，且当 includeSignature 为 true 时应包含 RSA SHA256withRSA 签名计算代码。

验证: 需求 3.1, 3.2, 3.3

Property 9: 签名生成与验证 Round-Trip

对于任意 appId、timestamp 和有效的 RSA 密钥对，使用私钥对签名字符串 `appId=[appId]&timestamp=[timestamp]` 生成签名后，使用对应公钥验证该签名应返回 true。

验证: 需求 4.1, 4.2

Property 10: 加密解密 Round-Trip

*对于任意*明文字符串和有效的 RSA 密钥对，使用公钥以 RSA/ECB/PKCS1Padding 算法加密后，使用对应私钥解密应得到原始明文。

验证: 需求 4.3

Property 11: 非法密钥格式错误处理

*对于任意*非法格式的密钥字符串（非 PEM 且非有效 Base64 编码的密钥），签名、验证和加密操作应返回明确的错误信息而非抛出未处理异常。

验证: 需求 4.4

Property 12: Webhook 配置 Round-Trip

*对于任意*有效的 Webhook 配置（包含 URL 和事件类型列表），保存后再查询应返回与原始配置等价的数据。

验证: 需求 5.2

Property 13: Webhook 日志完整性

对于任意 Webhook 推送事件，生成的日志记录应包含时间戳、事件类型、推送状态和重试次数四个必要字段。

验证: 需求 5.4

Property 14: 文件上传验证

*对于任意*文件上传请求，如果文件 MIME 类型不在允许列表（application/pdf、image/jpeg、image/png）中或文件大小超过 10MB，上传应被拒绝并返回明确的错误信息。

验证: 需求 6.2

Property 15: 入驻申请状态变更通知

*对于任意*入驻申请状态变更事件，系统应生成对应的通知记录。

验证: 需求 6.4

Property 16: 不完整申请提交拒绝

*对于任意*缺少必填字段的入驻申请数据，提交操作应被拒绝，且返回的错误信息应包含所有缺失的必填字段名称。

验证: 需求 6.5

Property 17: 订单筛选正确性

*对于任意*筛选条件组合（状态、支付方式、时间范围），返回的订单列表中每笔订单应同时满足所有指定的筛选条件。

验证: 需求 7.2, 7.3

Property 18: 订单详情完整性

*对于任意*订单 ID，查询返回的订单详情应包含状态流转历史和支付信息。

验证: 需求 7.4

Property 19: CSV 导出 Round-Trip

*对于任意*订单数据集，导出为 CSV 后再解析，应得到与原始数据等价的订单记录集合。

验证: 需求 7.5

Property 20: 环境切换端点映射

*对于任意*环境设置（生产或沙箱），平台中所有 API 端点地址和"在线试用"功能的请求目标应使用对应环境的域名（生产: openapi.osl-pay.com，沙箱: openapitest.osl-pay.com）。

验证: 需求 8.1, 8.4

Property 21: 角色权限控制

*对于任意*用户角色和任意受保护资源，如果该角色的权限列表中不包含该资源的访问权限，则访问请求应被拒绝；如果包含，则应被允许。

验证: 需求 9.2, 9.3

Property 22: 审计日志记录完整性

*对于任意*用户操作，系统应在审计日志中创建对应记录，包含操作时间、操作人、操作类型和操作结果。

验证: 需求 9.4

Property 23: MCP 工具签名自动处理

对于任意 MCP 工具调用（报价、创建订单、查询订单），MCP Server 应自动生成 RSA SHA256withRSA 签名并添加到请求头（X-App-Id、X-Timestamp、X-Signature），无需调用者手动处理签名逻辑。

验证: 需求 14

Property 24: MCP 工具调用顺序智能处理

*对于任意*创建订单的 MCP 工具调用，如果未提供 quoteId，MCP Server 应自动先调用报价接口获取 quoteId，再调用创建订单接口。

验证: 需求 14

Property 25: 结构化指南数据完整性

*对于任意*集成模式（WebSDK 或 OpenAPI），结构化指南 API 返回的数据应包含步骤列表、代码示例、常见问题和支持信息四个必要部分。

验证: 需求 16

Property 26: 结构化指南代码示例多语言支持

*对于任意*集成模式的结构化指南，代码示例部分应至少包含 Node.js、Python、Java 三种语言的实现。

验证: 需求 16

Property 27: MCP 配置环境感知

*对于任意*环境设置（生产或沙箱），MCP 配置中心生成的配置文件中的 `OSLPAY_ENV` 环境变量应与当前环境一致。

验证: 需求 15

错误处理

全局错误处理策略

错误类型	处理方式	HTTP 状态码
认证失败	返回错误信息，重定向到登录页	401
权限不足	返回权限不足提示	403
资源不存在	返回 404 错误	404
请求参数无效	返回字段级错误详情	400
文件格式/大小不合规	返回具体限制说明	422
密钥格式错误	返回期望格式说明（PEM/Base64）	400
外部 API 调用失败	记录日志，返回友好错误信息	502
服务器内部错误	记录日志，返回通用错误信息	500

错误响应格式

```
interface ErrorResponse {
  code: string;            // 业务错误码
  message: string;         // 用户友好的错误描述
  details?: {              // 字段级错误详情（用于表单验证）
    field: string;
    message: string;
  }[];
  traceId: string;         // 用于问题追踪的唯一标识
}
```

Webhook 重试策略

- 首次推送失败后，按指数退避策略重试（1s, 2s, 4s, 8s, 16s）
- 最多重试 5 次
- 5 次重试均失败后标记为 `final_failed`
- 所有推送尝试均记录到 Webhook 日志

测试策略

属性基测试（Property-Based Testing）

后端（Java）： 使用 `jqwik` 库（JUnit 5 平台上的属性基测试框架）实现后端服务的正确性属性验证。

前端/集成测试（Python）： 使用 `pytest` + `hypothesis` 库实现属性基测试。Hypothesis 是 Python 生态中最成熟的属性基测试框架，与 pytest 无缝集成。

配置要求：

- 每个属性测试至少运行 100 次迭代（Hypothesis 通过 `@settings(max_examples=100)` 配置）
- 每个测试用注释标注对应的设计文档属性编号
- 标注格式：`Feature: oslpay-merchant-portal, Property {N}: {属性标题}`

重点属性测试：

- Round-Trip 属性（Property 9, 10, 12, 19）：验证签名、加密、Webhook 配置和 CSV 导出的往返一致性
- 筛选正确性（Property 5, 17）：验证端点分类筛选和订单筛选的准确性
- 权限控制（Property 21）：验证角色权限映射的正确性
- 输入验证（Property 11, 14, 16）：验证错误输入的拒绝和错误信息

单元测试

后端： 使用 `JUnit 5` + `Mockito` 测试 Spring Boot 服务层和控制器。

前端： 使用 `vitest` + `@testing-library/react` 测试 React 组件和前端逻辑。

API 测试： 使用 `pytest` + `requests` 测试 REST API 接口。

覆盖以下场景：

- 特定 API 端点的解析结果验证（示例测试）
- 特定语言的代码生成输出验证
- 已知测试向量的签名计算验证
- Webhook 重试达到 5 次后的状态变更（边界条件）
- 特定文件格式和大小的上传验证
- 沙箱测试卡号的预期行为验证

集成测试

使用 `pytest` 编写端到端集成测试，后端使用 `Testcontainers` （MySQL、Redis）：

- API 文档"在线试用"功能与沙箱环境的端到端测试
- Webhook 测试推送的端到端验证
- 商户入驻申请提交流程的端到端测试
- 用户登录和权限控制的端到端测试